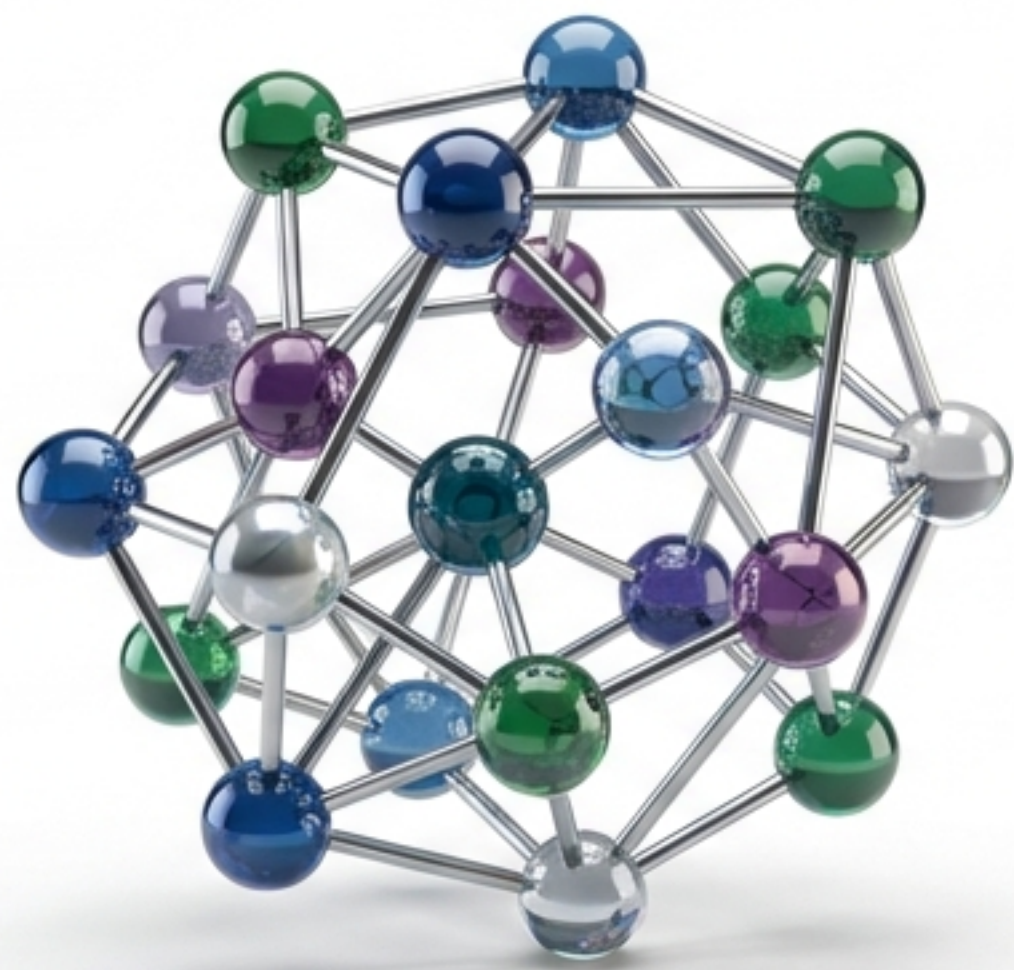




Machine Learning

Chương 8: Giảm chiều dữ liệu (Phần 3) - PCA Nâng cao

Tìm số chiều để giữ 95% phương sai

Chạy PCA mà không giảm
Chạy PCA mà không giảm chiều
để lấy toàn bộ các PCs, sau đó
tính toán lũy kế để tìm số chiều
d.



```
from sklearn.datasets import fetch_openml
import numpy as np

mnist = fetch_openml('mnist_784', as_frame=False)
X_train = mnist.data[:60_000]

pca = PCA()
pca.fit(X_train)

cumsum = np.cumsum(pca.explained_variance_ratio_)
d = np.argmax(cumsum >= 0.95) + 1
# d bang 154
```

Đặt `n_components` là tỷ lệ phần trăm

Một cách tốt hơn (và nhàn hơn) là truyền thẳng mức độ phương sai bạn muốn bảo toàn dưới dạng một số thập phân từ 0.0 đến 1.0 cho `n_components`.



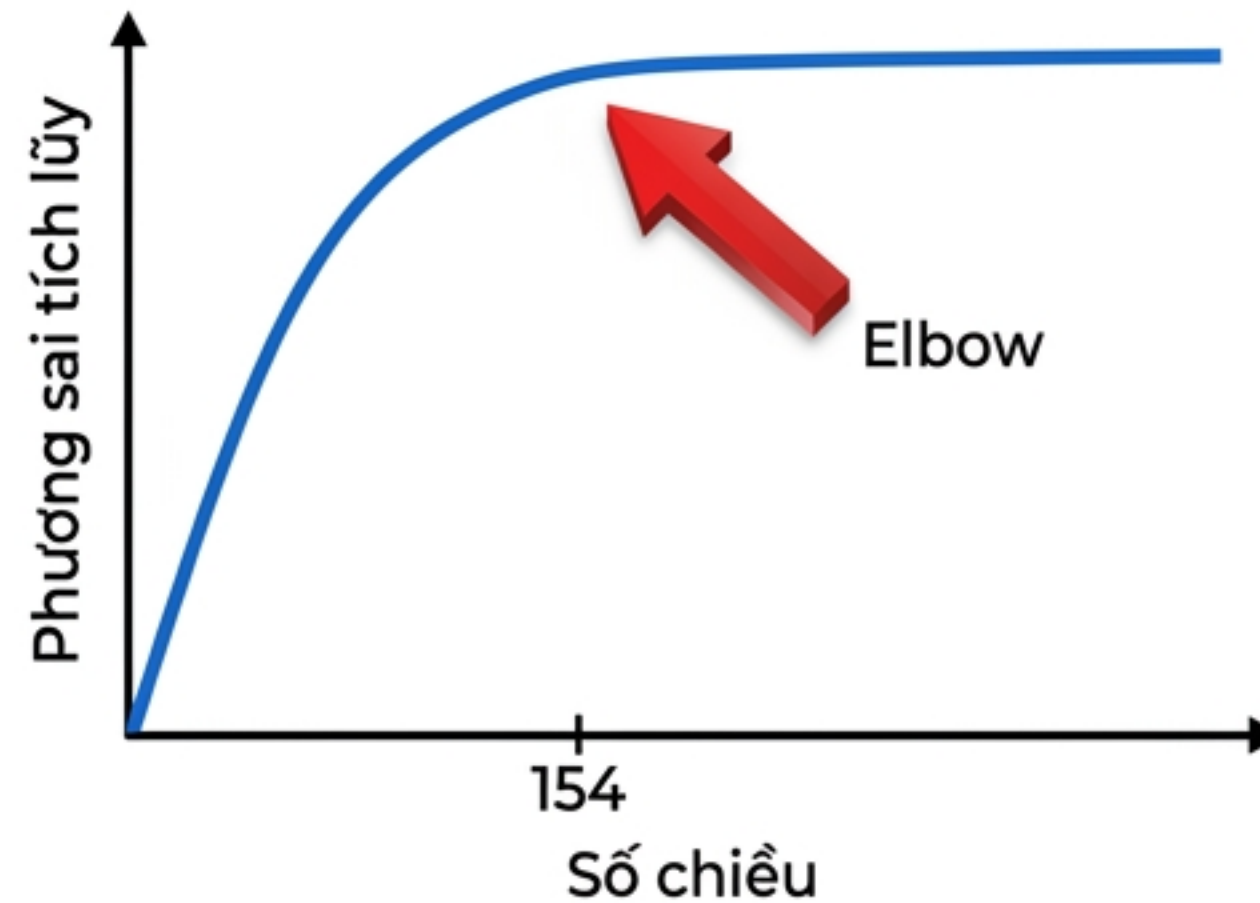
```
# Scikit-learn sẽ tự tính toán số chiều tối thiểu
pca = PCA(n_components=0.95)

X_reduced = pca.fit_transform(X_train)

print(pca.n_components_)
# Output: 154
```

Vẽ biểu đồ phương sai giải thích theo số chiều

Tại điểm uốn (elbow), việc tăng thêm số chiều không mang lại quá nhiều giá trị thông tin, giúp người phân tích dễ dàng quyết định số chiều cần thiết (đường đi ngang vì các chiều thêm vào không mang thêm phương sai).



Hình 8-8: Biểu đồ đường tăng dần và tạo thành một điểm uốn - elbow point - khi đạt $\approx 100-150$ chiều.

Điều chỉnh số chiều như một siêu tham số

- PCA thường là một bước tiền xử lý (preprocessing) cho một tác vụ Machine Learning khác.
- Số chiều d có thể được coi là một siêu tham số (Hyperparameter).
- Có thể dùng GridSearchCV hoặc RandomizedSearchCV để tìm số chiều tốt nhất cho bài toán tổng thể.



Sử dụng RandomizedSearchCV cho PCA

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV
from sklearn.pipeline import make_pipeline

clf = make_pipeline(PCA(random_state=42), RandomForestClassifier(random_state=42))
param_distrib = {
    'pca__n_components': np.arange(10, 80),
    'randomforestclassifier__n_estimators': np.arange(50, 500)
}
rnd_search = RandomizedSearchCV(clf, param_distrib, n_iter=10, cv=3, random_state=42)
rnd_search.fit(X_train[:1000], y_train[:1000])

print(rnd_search.best_params_)
# pca__n_components tối ưu chỉ là 23 thay vì 154
```



PCA dùng để nén dữ liệu (Compression)

784 → 154

(Giảm số đặc trưng)

≈ 20%

(Kích thước ban đầu)



**Chỉ mất 5%
thông tin!**

(Bảo toàn 95% phương sai)

Mức độ nén tuyệt vời giúp tiết kiệm bộ nhớ RAM và tăng tốc vượt trội thuật toán học máy phía sau.

Tái tạo dữ liệu (Giải nén)

Có thể giải nén từ 154 chiều ngược trở lại 784 chiều, nhưng đây là quá trình có hao hụt (lossy compression).

Không nhận lại được 100% pixel gốc do thiếu 5% phương sai đã loại bỏ. Hình ảnh sẽ bị mờ đi một chút.

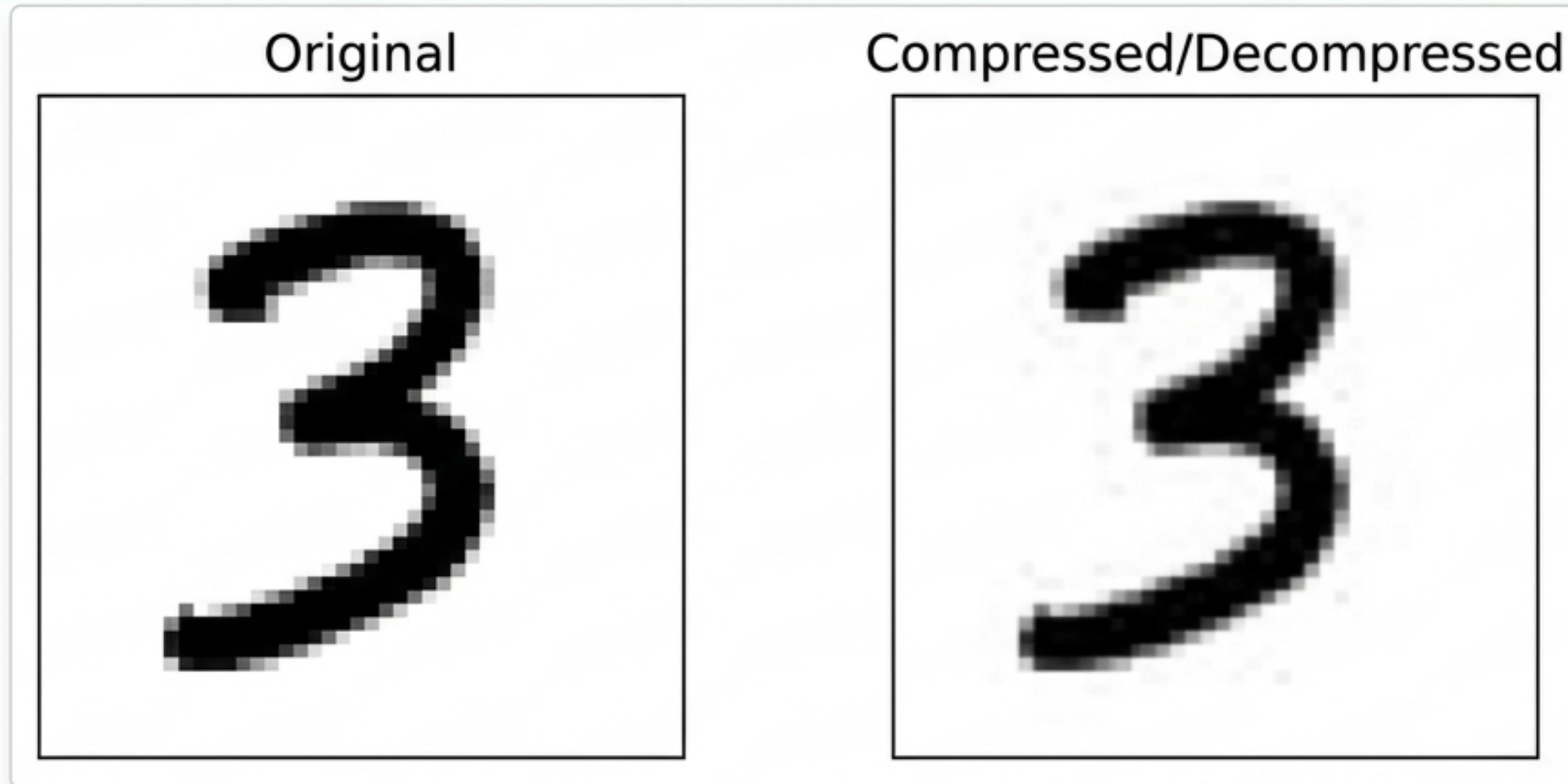
Sai số tái tạo (Reconstruction Error):

Khoảng cách bình phương trung bình giữa dữ liệu gốc và dữ liệu giải nén.



Phép biến đổi ngược (inverse_transform)

```
X_recovered = pca.inverse_transform(X_reduced)
```



Hình 8-9: Hình ảnh MNIST gốc so sánh với hình ảnh đã qua nén/giải nén bằng PCA.
Hình giải nén tuy không sắc nét hoàn hảo bằng nhưng vẫn dễ dàng đọc được là số mấy.

PCA ngẫu nhiên (Randomized PCA)

Dùng kỹ thuật ngẫu nhiên hóa để xấp xỉ tìm d thành phần chính đầu tiên một cách nhanh chóng. Nhanh hơn đáng kể khi $d \ll n$.

Thuật toán	Độ phức tạp tính toán
SVD Tiêu chuẩn	$O(m \times n^2) + O(n^3)$
Randomized PCA	$O(m \times d^2) + O(d^3)$



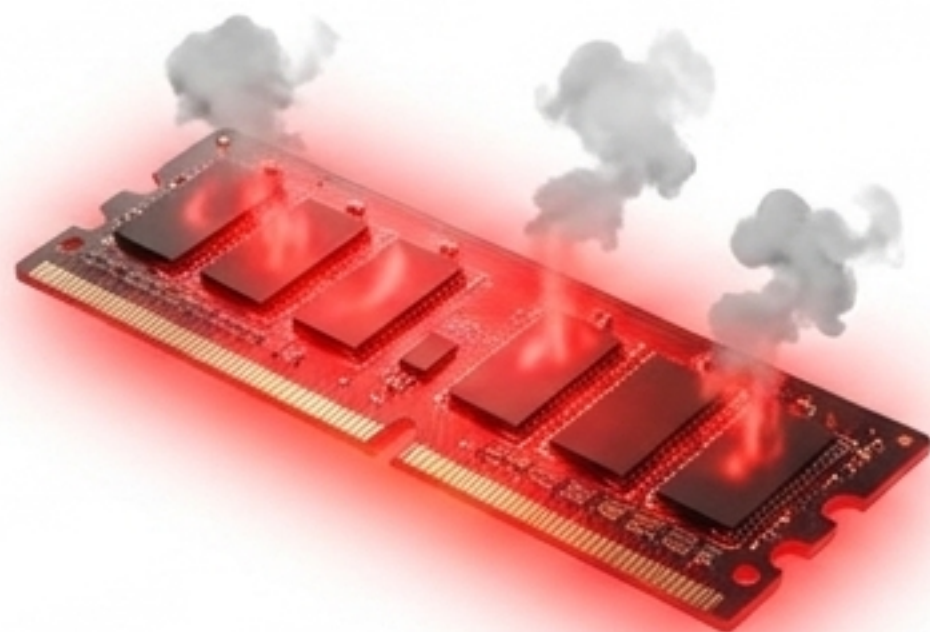
Cài đặt Randomized PCA

✓ Chỉ định tham số **svd_solver='randomized'**



```
rnd_pca = PCA(n_components=154, svd_solver='randomized', random_state=42)  
X_reduced = rnd_pca.fit_transform(X_train)
```

PCA tăng dần (Incremental PCA)



Vấn đề của PCA thông thường: Yêu cầu TOÀN BỘ tập dữ liệu phải nằm gọn trong bộ nhớ (RAM) cùng một lúc.



Giải pháp (IPCA): Chia tập huấn luyện thành các **mini-batch** và đẩy vào mô hình từng **lô nhỏ** một.

Ứng dụng: Hữu ích cho dữ liệu khổng lồ (Out-of-core learning) và **PCA thời gian thực** (Online).

Cài đặt Incremental PCA với array_split

Lưu ý gọi phương thức **partial_fit()** ở từng bước thay vì **fit()**.

```
from sklearn.decomposition import IncrementalPCA
import numpy as np

n_batches = 100
inc_pca = IncrementalPCA(n_components=154)

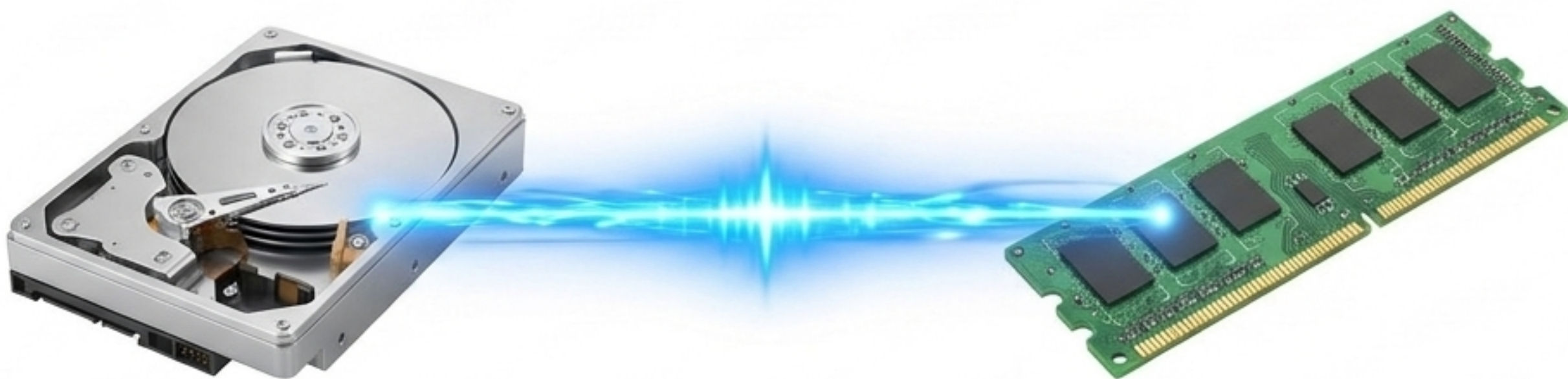
for X_batch in np.array_split(X_train, n_batches):
    inc_pca.partial_fit(X_batch)

# transform áp dụng cho toàn bộ
X_reduced = inc_pca.transform(X_train)
```



Tối ưu Incremental PCA với tệp bộ nhớ (mmap)

Dùng **np.mmap** để sử dụng file nhị phân trên đĩa như thể nó nằm trong RAM. Dữ liệu **chỉ tải** vào RAM khi cần.



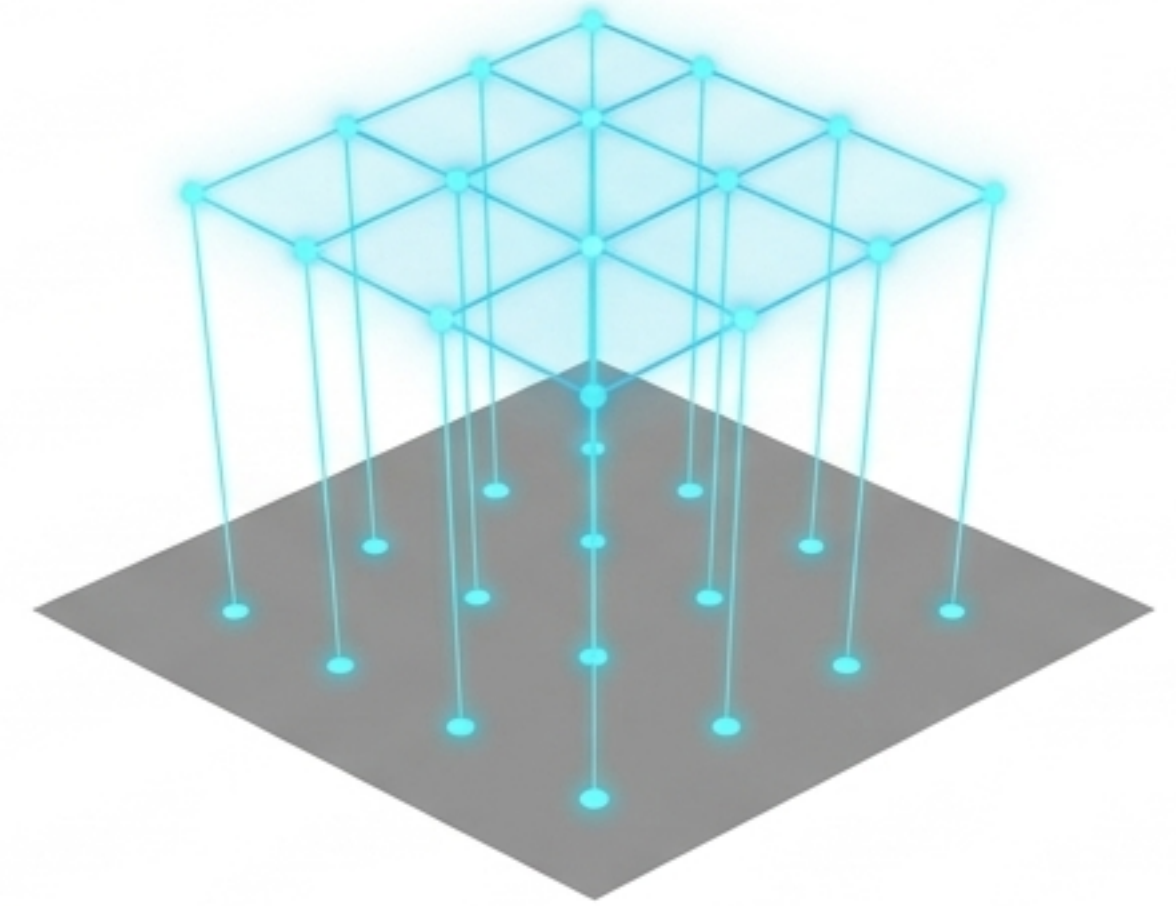
```
# doc file đã lưu dưới dạng mmap
filename = 'my_mnist.mmap'
X_mmap = np.mmap(filename, dtype='float32', mode='readonly').reshape(-1, 784)

batch_size = X_mmap.shape[0] // n_batches
inc_pca = IncrementalPCA(n_components=154, batch_size=batch_size)

# inc_pca hiểu cách hoạt động của mmap và lấy từng đoạn theo batch_size
inc_pca.fit(X_mmap)
```

Phép chiếu ngẫu nhiên (Random Projection)

Chiếu dữ liệu xuống không gian thấp bằng một ma trận ngẫu nhiên. Bảo toàn khoảng cách mà không cần huấn luyện. Rất nhẹ và cực nhanh cho dữ liệu chiều cực lớn.



Định lý Johnson-Lindenstrauss (JL)

- Chiếu m điểm xuống không gian d chiều sao cho khoảng cách bình phương chỉ méo mó tối đa sai số ϵ .
- Ví dụ: $m = 5000$ điểm, $\epsilon = 0.1 \rightarrow d \approx 7300$ chiều là an toàn.
- Điểm mấu chốt: Định lý này không quan tâm đến số chiều gốc n (ngay cả khi $n = 1$ triệu).